```
  ) {
    return new Book(
      null, isbn, title, author, price, publisher, null, null, 0
    );
  }
}
```

❶ A new, optional field

NOTE After making this change, you must update the classes that call the static factory method and the `Book()` constructor to include a value for the `publisher` field. You can use either `null` (since it's optional) or a string value like `Polarsophia`. Check the source code (Chapter05/05-end/catalog-service) to see the final result. In the end, check that both the autotests and the application run correctly.

When this new version of Catalog Service is deployed to production, Flyway will skip the V1__Initial_schema.sql migration because it's already been applied, but it will execute the change described in V2__Add_publisher_column.sql. At this point, bookshop employees can start including the publisher name when adding new books to the catalog, and all the existing data will still be valid.

What if you need to make the `publisher` field mandatory? You can do that in a third version of Catalog Service, using an SQL migration to enforce the `publisher` column to be `NON NULL` and implementing a Java migration that adds a publisher to all the existing books in the database that don't have one already.

This two-step approach is very common to ensure backward compatibility during an upgrade. As you'll learn in a later chapter, there are usually multiple instances of the same application running. Deploying a new version is generally done through a *rolling upgrade* procedure consisting of updating one (or a few) instances at a time to ensure zero downtime. There will be both old and new versions of the application running during the upgrade, so it's paramount that the old instances can still run correctly even after the database changes introduced in the latest version have been applied.